

Geovisualization of OpenStreetMap Data in Berlin

Where are the restaurants, cafés, pharmacies ... and where are the gaps?

Tools: Python · pandas · GeoPandas · OSMnx / Overpass API · Folium · Streamlit · GitHub

1. Background

OpenStreetMap (OSM) is the free world map created by volunteers. It contains millions of points of interest (POIs) such as restaurants, cafés, pharmacies, supermarkets or playgrounds. Every object carries key=value tags, for example amenity=restaurant or cuisine=italian.

You build a Streamlit app in which the user chooses in pull-down menus which OSM data source is shown and how it is aggregated and displayed. The guiding question is:

Guiding question

Which parts of Berlin are well supplied with a given type of place, where are hotspots and gaps, and how reliable is the OSM data behind this conclusion?

Important: OSM is crowd-sourced and its completeness varies. An area with few points may simply be less well mapped. Your analysis has to take this into account.

2. Data

Dataset	Content	Source
OSM points of interest	One file per category, e.g. all amenity=restaurant in Berlin	OpenStreetMap via Overpass API (OSMnx or Overpass Turbo)
geodata_berlin_plz.csv	Postal code (PLZ) polygons as WKT, WGS84	Project template
geodata_berlin_dis.csv	District (Bezirk) polygons as WKT, WGS84	Project template
plz_einwohner.csv	Columns plz, note, einwohner, qkm, lat, lon	suche-postleitzahl.org/downloads

Licence: OSM data is under the ODbL. The app must show “© OpenStreetMap contributors”.

The app needs at least six data sources, at least one outside gastronomy, plus one of your own choice. Suggestions (more tags: wiki.openstreetmap.org, “Map features”):

Menu label	OSM tag	Category attribute	Comment
Restaurants	amenity=restaurant	cuisine	Values separated by “;”
Cafés	amenity=cafe	cuisine / brand	Chains vs. independent
Fast food	amenity=fast_food	cuisine	Compare with restaurants
Pharmacies	amenity=pharmacy	brand	Basic supply
Supermarkets	shop=supermarket	brand	Discounters vs. organic

Menu label	OSM tag	Category attribute	Comment
Playgrounds	leisure=playground	access	Often mapped as polygons
Charging stations	amenity=charging_station	operator	Electric mobility
Own choice	team decides	team decides	Justify in documentation

3. The target application

One Streamlit page with a sidebar and a map. All settings are pull-down menus (st.selectbox).

#	Menu	Options	Behaviour
1	Data source	All labels from config.py	Decides which file is loaded (cached).
2	Category filter	"All" plus the ten most frequent values of the category attribute	Depends on menu 1; the list changes with the data source.
3	Area unit	Postal code (PLZ) / District (Bezirk)	Polygons used for aggregation.
4	Indicator	Number / Number per km ² / Number per 10,000 residents	Residents are only available per PLZ (see Task 4).
5	Map type	Choropleth / Heatmap / Point map with clusters	Choropleth uses menus 3 and 4; the others show single points.

Below the map: key figures (number of objects, number of empty areas, top three areas), an expandable data table with CSV download, the data quality report, and the attribution with the download date.

4. Tasks

Task 1 – Download the OSM data

Write a separate script `download_osm.py` that downloads one file per data source. The app must not call the Overpass API at runtime: download once and commit the files to datasets/.

- Use OSMnx (`ox.features_from_place`) or Overpass Turbo with GeoJSON export.
- OSM objects can be nodes or polygons. Convert every geometry to one point: compute the centroid in EPSG:25833, then transform back to EPSG:4326.
- Keep OSM type, id, name, lat, lon, the category attribute and `addr:postcode`. Not every tag exists, so select columns defensively. Save as UTF-8 CSV (`sep=";"`).

Example Overpass query:

```
[out:json][timeout:120];
area["name"="Berlin"]["boundary"="administrative"]["admin_level"="4"]->.a;
nwr["amenity"="restaurant"](area.a);
out center tags;
```

Task 2 – Set up and configure

- Unzip the starter package, create your GitHub repository and a virtual environment, and install `requirements.txt`. The polygon and residents files are already in `datasets/`.
- Define all data sources in `config.py`. The menu must be generated from this dictionary, so a new source needs only one entry and one download (see appendix).

Task 3 – Preprocessing and spatial join

- Create a GeoDataFrame with Point geometry (EPSG:4326); load the polygons from WKT and set the CRS explicitly.
- Assign every point to a PLZ and a Bezirk with `geopandas.sjoin` (predicate="within"). Do not rely on `addr:postcode`, which is missing for many objects. Report how many objects have it and how often it matches the spatial join.
- Remove duplicates created by points on borders; split multi-valued attributes (`cuisine=italian;pizza` counts for both).
- Aggregate per area. Start from the polygons so that areas without objects appear with 0.

Task 4 – Indicators

- Number per km²: compute the area in EPSG:25833, never in degrees.
- Number per 10,000 residents: use `einwohner` from `plz_einwohner.csv`, restricted to Berlin postal codes. For Bezirke, sum the PLZ values via a spatial assignment of the PLZ centres, or disable the indicator with a hint.
- Handle division by zero and very small populations, and document your decision.

Task 5 – Data quality check

Write `quality_report()`, shown in an expander "Data quality". It checks at least: number of objects; share without name and without category attribute; near-duplicates (same name, less than 10 m apart); points outside Berlin; plausible coordinates (lat about 52.3–52.7, lon about 13.0–13.8); consistency of `addr:postcode` with the spatial join; number of PLZ areas without objects.

Task 6 – Streamlit application

- Implement the five pull-down menus in the sidebar; make menu 2 dependent on menu 1 and handle sources without values in the category attribute.
- Implement the map types with Folium: choropleth with legend, `folium.plugins.HeatMap`, and `folium.plugins.MarkerCluster` with the name as tooltip.
- Use `@st.cache_data` for loading and joining; show a clear message when a selection returns no data.

Task 7 – Run, publish, deploy

- Run locally with the command below and test every combination of menus 1, 3 and 4 at least once.
- Push code, config and datasets to GitHub with a short README, then deploy on Streamlit Community Cloud (streamlit.io). Use relative paths instead of an absolute working directory. Keep each dataset well below 100 MB.

```
streamlit run main.py
```

Task 8 – Analysis

Answer with screenshots and the menu settings used:

- Which Bezirk and PLZ have the most restaurants in absolute numbers, and which per 10,000 residents? Explain the difference.
- Compare two data sources: where do the patterns agree or differ, and why?
- Where do you see a supply gap for a source of your choice, and what further information would you need to be sure?
- Where might the result be distorted by mapping quality? Use your data quality report.

Task 9 – Documentation

- Program: repository structure, purpose of files and functions, data flow from download to map.
- Data: queries, download date, preprocessing steps and reasons.
- Results: answers to Task 8 and a critical discussion of limits (OSM completeness, choice of area unit, normalisation). Teams state who did what.

Additional tasks (not evaluated)

- Show two data sources as separate layers with a layer control and compute their ratio per area (e.g. pharmacies per supermarket).
- Add a dropdown for the tag “wheelchair” (yes / limited / no) and analyse accessibility.
- Compute the distance from each PLZ centre to the nearest object of the selected source as an alternative indicator.

5. Deliverables and grading

Part	Deliverable	Points
Data download (Task 1)	download_osm.py, CSV files	10
Preprocessing, spatial join, indicators (Tasks 2–4)	core/methods.py, config.py	25
Data quality check (Task 5)	quality_report(), output in app	10
Streamlit app (Task 6)	main.py, working locally	20
GitHub and deployment (Task 7)	Repository and app links	10
Analysis (Task 8)	Answers with screenshots	15
Documentation and code quality (Task 9)	PDF, readable code	10
Total		100

6. Common pitfalls

- Coordinate order: GeoPandas uses (lon, lat), Folium expects [lat, lon].
- Areas and distances must never be computed in degrees; use EPSG:25833.
- A linear colour scale is dominated by outliers; test a quantile-based or logarithmic scale.
- Central districts are mapped more densely; a low value is no proof of a lack of supply.
- Do not send Overpass requests in a loop; thousands of separate folium.GeoJson objects are slow, so use one layer with a style function.

Appendix – Code skeleton

Excerpt of the starter package (complete files are provided). Lines with TODO are for you to implement.

config.py

```
p = dict()
p["geocode"] = "PLZ"
p["file_plz"] = "geodata_berlin_plz.csv"
p["file_dis"] = "geodata_berlin_dis.csv"
p["file_residents"] = "plz_einwohner.csv"

# menu label -> OSM tags, file, attribute for the category filter
p["sources"] = {
    "Restaurants": {"tags": {"amenity": "restaurant"},
                    "file": "osm_restaurants.csv", "subcat": "cuisine"},
    "Pharmacies": {"tags": {"amenity": "pharmacy"},
                   "file": "osm_pharmacies.csv", "subcat": "brand"},
    # TODO: add at least four more sources
}
p["areas"] = {"Postal code (PLZ)": "PLZ", "District (Bezirk)": "Bezirk"}
p["indicators"] = ["Number", "Number per km2", "Number per 10,000 residents"]
p["maptypes"] = ["Choropleth", "Heatmap", "Point map (clusters)"]
pdict = p.copy()
```

download_osm.py

```
import osmnx as ox
from config import pdict

KEEP = ["name", "cuisine", "brand", "operator", "access", "addr:postcode"]

def download_source(label, cfg):
    gdf = ox.features_from_place("Berlin, Germany", tags=cfg["tags"])
    gdf = gdf.reset_index()
    # TODO: centroid in EPSG:25833, back to EPSG:4326, add lat / lon
    # TODO: keep osm type, id, lat, lon and the existing columns from KEEP
    # TODO: write to "datasets/" + cfg["file"] (sep=";", encoding="utf-8")

if __name__ == "__main__":
    for label, cfg in pdict["sources"].items():
        download_source(label, cfg)
```

core/methods.py and main.py

```
@st.cache_data
def load_areas(file, key):
    # WKT csv -> GeoDataFrame, EPSG:4326
    # TODO

@st.cache_data
def load_osm_points(file):
    # csv -> GeoDataFrame of Points
    # TODO

def assign_area(gdf_points, gdf_areas, key):
    # gpd.sjoin, no duplicates
    # TODO

def aggregate(gdf_points, gdf_areas, key, indicator, residents):
```

```

    # TODO: ALL areas with column 'value' (0 if empty)

def quality_report(gdf_points, gdf_areas, subcat=None):
    # TODO

def make_map(gdf_points, gdf_agg, maptype, key, colors, caption):
    # TODO

def make_streamlit_app(pdct, df_residents):
    st.title("OpenStreetMap Berlin: Places and Supply")
    src = st.sidebar.selectbox("Data source", list(pdct["sources"].keys()))
    cat = st.sidebar.selectbox("Category filter", ["All"]) # TODO: depends on src
    area = st.sidebar.selectbox("Area unit", list(pdct["areas"].keys()))
    ind = st.sidebar.selectbox("Indicator", pdct["indicators"])
    mtype = st.sidebar.selectbox("Map type", pdct["maptypes"])
    # TODO: load, filter, aggregate, map, key figures, quality report

# main.py
def main():
    df_residents = # read plz_einwohner.csv, m1.preprop_resid(...)
    m1.make_streamlit_app(pdct, df_residents)

```